Quantum Machine Intelligence (2024) 6:89
https://doi.org/10.1007/s42484-024-00222-8

RESEARCH ARTICLE

Coherent feed-forward quantum neural network

Utkarsh Singh[1,2] · Aaron Z. Goldberg[1,2] · Khabat Heshami[1,2,3]

Received: 26 January 2024 / Accepted: 6 November 2024 / Published online: 3 December 2024
© Crown 2024

Abstract
Quantum machine learning, focusing on quantum neural networks (QNNs), remains a vastly uncharted field of study. Current QNN models primarily employ variational circuits on an ansatz or a quantum feature map, often requiring multiple entanglement layers. This methodology not only increases the computational cost of the circuit beyond what is practical on near-term quantum devices but also misleadingly labels these models as neural networks, given their divergence from the structure of a typical feed-forward neural network (FFNN). Moreover, the circuit depth and qubit needs of these models scale poorly with the number of data features, resulting in an efficiency challenge for real-world machine learning tasks. We introduce a *bona fide* QNN model, which seamlessly aligns with the versatility of a traditional FFNN in terms of its adaptable intermediate layers and nodes, absent from intermediate measurements such that our entire model is coherent. This model stands out with its reduced circuit depth and number of requisite CNOT gates, achieving a more than 50% reduction in both compared to prevailing QNN models. Furthermore, the qubit count in our model remains unaffected by the data's feature quantity. We test our proposed model on various benchmarking datasets such as the breast cancer diagnostic (Wisconsin) and credit card fraud detection datasets. Our model achieved an accuracy of 91% on the breast cancer dataset and 85% on the credit card fraud detection dataset, outperforming existing QNN methods by 5–10% while requiring approximately 50% fewer quantum resources. These results showcase the advantageous efficacy of our approach, paving the way for the application of quantum neural networks to relevant real-world machine learning problems.

Keywords Quantum machine learning · Quantum neural networks · Quantum computation · Coherent feed forward

1 Introduction

Over the past decade, quantum machine learning (QML) (Schuld and Petruccione 2021) has emerged as a dynamic field with promising potential for advancing machine learning techniques using quantum computing, especially because quantum machines may efficiently explore large-dimensional spaces for processing large amounts of data (Nielsen and Chuang 2011; Biamonte et al. 2017; Schuld et al. 2014; Zoufal et al. 2021; Havlicek et al. 2019; Garcia et al. 2022). While initial research focused primarily on adapting standard machine learning algorithms to quantum computing, such as quantum neural networks (QNNs) (Diep 2020; Chalumuri et al. 2021; Chen and Niu 2020; Tacchino et al. 2021; Wang et al. 2019; Li et al. 2020), quantum generative adversarial networks (QGANs) (Situ et al. 2018; Zoufal et al. 2019), and quantum support vector machines (Havlicek et al. 2019; Wu et al. 2021), progress has been hindered by the lack of a clear path to scalability and practical applications of these methods. Instead, researchers have focused on developing algorithms suitable for the current generation of noisy intermediate-scale quantum (NISQ) devices (Wan et al. 2017; Beer et al. 2020; Bondarenko and Feldmann 2020; Cong et al. 2019; Li et al. 2020; Sharma et al. 2022; Zhou et al. 2023; Bharti et al. 2022), resulting in a new wave of algorithms known as NISQ-era QML algorithms (Preskill 2018).

These recent QML algorithms are predicated upon low-depth parameterized quantum circuits (Sim et al. (2019); Zhang et al. 2023), which take a hybrid approach that combines the strengths of quantum processors with classical

✉ Khabat Heshami
khabat.heshami@nrc-cnrc.gc.ca

1 National Research Council of Canada, 100 Sussex Drive, Ottawa, Ontario K1N 5A2, Canada

2 Department of Physics, University of Ottawa, 25 Templeton Street, Ottawa, Ontario K1N 6N5, Canada

3 Institute for Quantum Science and Technology, Department of Physics and Astronomy, University of Calgary, 25 Templeton Street, Calgary, Alberta T2N 1N4, Canada

processors. This hybridization allows for the development of novel algorithms that have shown some advantages in specific use cases (Abbas et al. 2021; Biamonte et al. 2017; Jiang et al. 2021; Yamasaki et al. 2023), although a useful quantum advantage remains to be seen.

Most QML models currently available employ complex encoding techniques, known as quantum feature maps, and use parameterized quantum circuits as models (Garcia et al. 2022; Havlicek et al. 2019) in place of the intermediate layers of a neural network. Sometimes, these circuits are concatenated such that the measurement of one circuit provides nonlinearity when inputting data into the subsequent circuit (Tacchino et al. 2020). In the hybrid approach, post-processing steps are similar to those used in classical machine learning (ML), updating the parametrized quantum circuits using techniques such as gradient descent, and are performed on a classical computer. Nonetheless, these feature mapping techniques often stumble when faced with real-world datasets, as the requisite number of qubits and the circuit depth escalate with the number of features in the data.

In this work, we propose a novel circuit-based approach that incorporates entanglement layers for the connections between nodes in adjacent layers, resembling a classical feed-forward neural network (FFNN). This approach offers the advantage of adaptable intermediate layers, allowing us to adjust them according to the characteristics of the data, which is particularly significant for classification tasks. Further, because all of the hidden layers can be incorporated without intermediate measurements, our approach is fully coherent throughout the evolution of the circuit and can take advantage of quantum coherence properties throughout, which is responsible for quantum advantages in related settings (Gyurik et al. 2023). Finally, by developing a data encoding scheme inspired by classical neural networks that writes multiple features onto a small number of qubits, our overall use of quantum resources is amenable to quantum computers available today.

To evaluate the effectiveness of our model, we conduct numerical experiments using the credit card fraud detection and Wisconsin breast cancer diagnostic datasets, employing the Qiskit package for simulations of quantum circuits, and compare the results to state-of-the-art QNN models. The results of these experiments are presented and show that our approach achieves significant improvements in both accuracy and computational efficiency over traditional QNN methods.

Our results highlight the potential of integrating classical neural network concepts into quantum computing frameworks, opening avenues for more sophisticated, resource-efficient quantum models in the future. As we continue to explore and refine our model that we dub the coherent feed-forward quantum neural network (CFFQNN), we anticipate its adaptability to a broader range of applications, further bridging the gap between quantum computing and real-world machine learning challenges.

1.1 Artificial neural network

The functioning of the brain inspires the idea of an artificial neural network (ANN). The brain receives and processes information via a network of neurons, where each neuron receives inputs from a number of neurons, processes them, and produces an output that is then input to subsequent neurons. In an ANN, perceptrons or nodes are used to mimic biological neurons: each is linked to others with variable weights, and the structure of the connections between nodes and their weights can then be used to process data; one artificial neuron is shown in Fig. 1.

Perceptrons were introduced by Rosenblatt (1957) as binary classifiers that form the foundational concept behind artificial neural networks and deep learning. A perceptron takes multiple input values X_i and produces a single binary-outcome output y. Each input is associated with a weight W_i, which signifies the importance of that input. The perceptron computes a weighted sum of its inputs with an overall bias b via

$$z = \sum_i W_i X_i + b \tag{1}$$

and passes this sum through an activation function, typically a step function, to produce its output y:

$$y = \sigma(z). \tag{2}$$

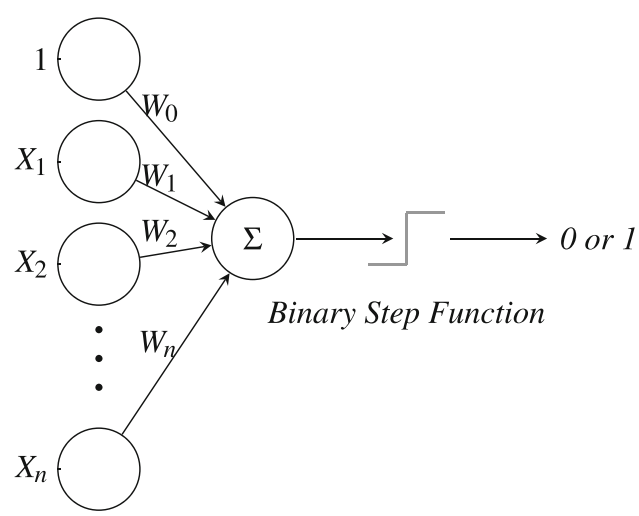

Fig. 1 A perceptron, inspired by neural networks in the brain. Inputs $\{X_i\}$ are combined with weights $\{W_i\}$ including a bias W_0 that are processed nonlinearly to produce a binary output

Here, to make it a differentiable non-linear activation function, σ is typically taken to be a logistic sigmoid or rectified linear unit function. If the weighted sum exceeds a certain threshold, the perceptron outputs a 1 (or "active"); otherwise, it outputs a 0 (or "inactive").

The perceptron's strength lies in its simplicity, adept at modelling linearly separable data, but it falters with non-linear data. This spurred the evolution of multi-layer perceptrons (MLPs) or neural networks capable of handling complex, non-linear patterns. ANNs optimize weights in each layer using methods like backpropagation and gradient descent, allowing them to approximate any function with high accuracy (Cybenko 1989; Hornik 1991). FFNNs, a key type of ANN, allow unidirectional information flow and have shown impressive performance in tasks like classification and regression. Any exemplary FFNN will be shown below in Fig. 5a.

1.2 Quantum neural network

Quantum neural networks are among the most popular algorithms in QML. Even though the field is not fully developed, there is a rapidly growing set of people exploring potential quantum advantages (Sagingalieva et al. 2023; Bondarenko and Feldmann 2020; Jiang et al. 2021; Farhi and Neven 2018).

The momentum behind recent advancements in this domain can largely be attributed to the variational techniques prevalent in numerous hybrid algorithms (Garcia et al. 2022). In general, one starts by encoding the classical data $\mathbf{X} \in \mathbb{R}^N$ onto the quantum state using a feature map $U(\mathbf{X})$ that can be implemented by a quantum circuit (Asfaw et al. 2020). Next, a variational circuit $U_{var}(\boldsymbol{\theta})$ is applied with intelligently chosen single-qubit rotations $R(\theta_i)$ and entanglement layers on an input state $U(\mathbf{X})|0\rangle^{\otimes n}$, where the θ_i parameters are the trainable weights and n is the number of qubits employed and usually scales linearly with N. Finally, the expectation value of some observable (e.g., $\hat{Z}^{\otimes n}$) is measured for the classical post-processing to predict the class $\tilde{y}$ of the input data. Optimizing the weight parameters present in the variational circuit is done using some classical optimizers, eventually minimizing the cost function $C(y(\mathbf{X}, \boldsymbol{\theta}), \tilde{y})$ in consideration (Garcia et al. 2022). A schematic diagram of a typical QNN is shown in Fig. 2.

While this approach of QNNs demonstrates promising outcomes in certain applications, it faces substantial challenges in scalability and gate complexity, particularly on NISQ devices (Cerezo et al. 2022; Beer et al. 2020). The number of qubits required in QNNs tends to increase linearly with the number of data features, quickly exceeding the limited qubit capacity of current quantum hardware and thus restricting their applicability to smaller datasets, instead of making use of the exponential scaling of Hilbert space dimension with number of qubits. Additionally, the number of (CNOT) gates, crucial for creating entanglements in quantum circuits, scales nearly quadratically with the number of features. This scaling is problematic on NISQ devices due to increased error rates and circuit depth, leading to higher likelihoods of decoherence and computational inefficiency. The combination of these scalability issues with the gate complexity challenges significantly hinders the practical implementation of QNNs on existing quantum platforms, making the handling of complex, feature-rich datasets a formidable task and posing a significant bottleneck in fully leveraging quantum computing for advanced machine learning applications.

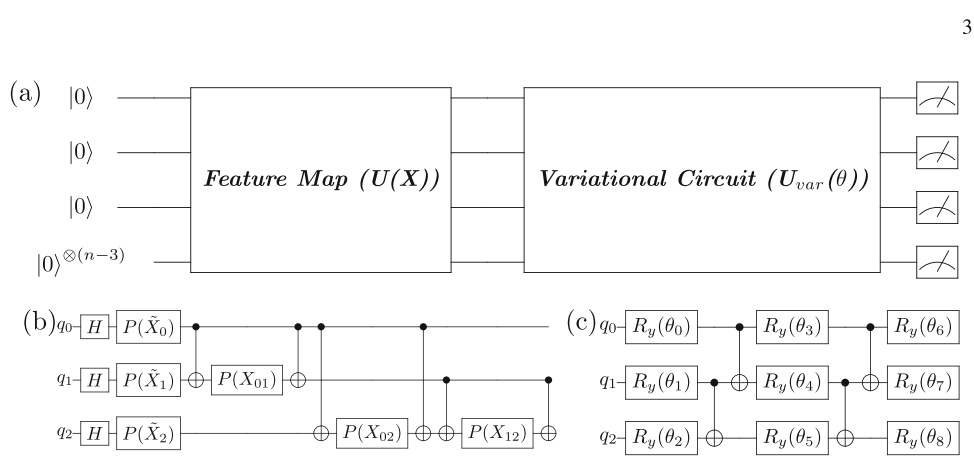

Fig. 2 **a** Architecture of a QNN acting on n qubits; the data $\mathbf{X}$ are loaded with a feature map $U(\mathbf{X})$, and the data are processed using a parametrized circuit $U_{var}(\theta)$. Subsequent measurement allows the parameters θ to be optimized and updated. These steps may be repeated. **b** A 3-qubit feature map circuit administering the commonly used ZZFeatureMap. Here, H represents the Hadamard gate, P represents the phase gate, $\tilde{X}_i = 2X_i$, and $X_{ij} = 2(\pi - X_i)(\pi - X_j)$. **c** A 3-qubit variational circuit with weight parameters $\{\theta_j\}$ explicit

2 Results

2.1 The model: coherent feed-forward quantum neural network

Here, we introduce our CFFQNN model that uses a quantum-classical hybrid approach to process data. The source code for all of our work can be found on GitHub as detailed below. The initial encoding layer is similar to the first hidden layer of a conventional ANN, as illustrated in Fig. 3, with the classical data loaded onto quantum states. Subsequent layers consist of a network single-qubit and controlled-rotation (entangling) gates, all of which are parametrized by their rotation angles, exemplified in Fig. 4. Ultimately, the qubits undergo measurement. Notably, the parameterized controlled rotations are adaptable to cater to specific network demands, and the measurement process can be tailored based on both the data and the desired structural outcome. This circuitry inherently mirrors the architecture of an ANN, as can be seen by comparing Fig. 5 a versus b.

We elect to perform all rotations about the y-axis for reasons that will become clear shortly. These are mathematically described by the single-qubit rotation gate expressed in the single-qubit computational basis $\{|0\rangle, |1\rangle\}$ as

$$R_y(\theta) = \exp(-\mathrm{i}\theta\sigma_y/2) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$$

using the Pauli matrix σ_y. Sequential rotations about the same axis commute and act additively as $R_y(\theta_1)R_y(\theta_2) = R_y(\theta_1 + \theta_2)$.

In the first layer, the data points with weights are encoded as the rotation angle of the R_y gate acting on some initial state, where the latter is taken to be some general state $R_y(b)|0\rangle$. All of the data points are successively encoded onto the same qubit, which is schematized in Fig. 3. Since all of the rotations are about the same axis, such an encoding can be performed with a single single-qubit gate parametrized by $\theta = z = \sum_{i=1}^{N} X_i W_i + b$:

$$R_y(X_N W_N) \cdots R_y(X_2 W_2) R_y(X_1 W_1)[R_y(b)|0\rangle] = R_y(z)|0\rangle. \tag{3}$$

This is the first efficiency resulting from all of the rotations being about the same axis, which reduces the number of data

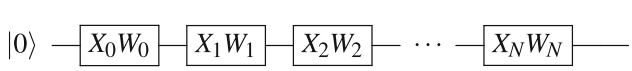

Fig. 3 The depiction of data encoding stage. Rotation gates with angles X_0W_0 act on a single qubit in analogy with inputs acting on a single neuron. An extra rotation gate with $X_0 = 1$ and $W_0 = b$ is added for flexibility to bias the initial qubit

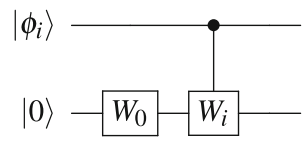

Fig. 4 An illustration of a single intermediate node in CFFQNN and how the weight values are applied from one node to the next using controlled rotations by angle W_i, with a possible single-qubit rotation by bias angle W_0

encoding gates, and also is responsible for better mimicking an ANN by directly encoding the variable z without giving a preference to the *ordering* among nodes within a given layer.

This data encoding procedure can be repeated on multiple distinct qubits to allow for more non-linear processing of the data in the quantum circuit. Therefore, a CFFQNN with n qubits in the first layer can be described as

$$|\Psi\rangle = [R(z)|0\rangle]^{\otimes n}. \tag{4}$$

Such single gates are exemplified in the "1st hidden layer" segment of Fig. 5b.

Since n is, in principle, independent from N, the size of the circuit need not to depend on the number of data features, unlike the case for a traditional QNN.

In the subsequent layers corresponding to hidden layers of an ANN, we initialize new qubits by applying a parametrized rotation gate as a biasing term $R_{y;\,j}(\theta_{0j})$ acting on the jth qubit, and then we apply parametrized controlled-rotation gates that are controlled by the qubits in the first layer. For example, if we want to connect the first node in the first layer to the first node in the second layer, we apply a rotation on the $(n+1)$th qubit controlled by the state of the *1*st qubit:

$$CR_y^{1\mapsto n+1}(\theta) = |0\rangle_1\langle 0| \otimes \mathbb{I}_{n+1} + |1\rangle_1\langle 1| \otimes R_{y;\,n+1}(\theta). \tag{5}$$

The other nodes are similarly connected by controlled-rotation gates $CR_y^{i\mapsto j}(\theta_{ij})$, each parametrized by independent weights θ_{ij}. Even though the rotations are controlled by different control qubits in different states, all of the rotations on a new qubit commute, such that the aggregate effect on a node in a hidden layer is independent from any ordering among the nodes in the previous layer.

Figure 4 displays a single intermediate node in the CFFQNN's hidden layer. It illustrates how the node applies weight values (denoted as W) to these nodes depending on the value of the last layer nodes. After this operation, the state $|\phi_i\rangle \otimes |0\rangle$ changes to a new state $|\psi_{ij}\rangle = CR_y^{i\mapsto j}(\theta_{ij})|\phi_i\rangle \otimes |0\rangle$ that may continue to be acted upon by other control layers.

While a classical ANN uses a perceptron to decide whether or not a certain node should forward its output to the next one, the CFFQNN retains the branches of the wavefunction associated with the control qubit being in each of states $|0\rangle$ and $|1\rangle$ without collapsing the state via measurement. For

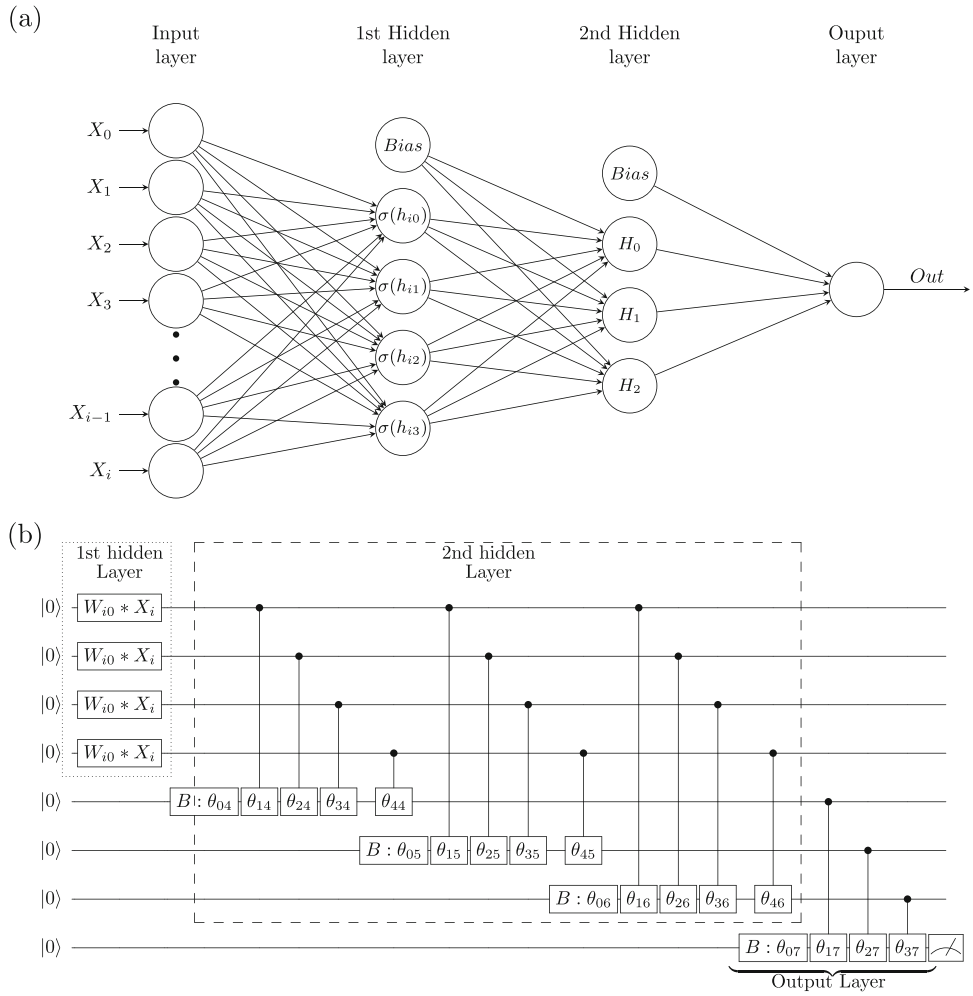

Fig. 5 **a** Architecture of an artificial neural network with two hidden layers. Here, W represents the weight parameters, $\mathbf{X}$ are data points, σ is a non-linear activation function, and $h_{ij} = W_{ij} X_i$. **b** Architecture of a CFFQNN with two hidden layers where $\mathbf{X}$ are data points, and $\mathbf{W}$ and $\boldsymbol{\theta}$ represent the weight parameters. The number of modes in a layer of the ANN corresponds to the number of qubits in a layer of the CFFQNN. In contrast to earlier QNN models such as those in Fig. 2c, the parameters of the CFFQNN change the controlled operations such that the CFFQNN circuits are not solely parametrized by their single-qubit gates; this is what allows the CFFQNN to resemble an ANN

example, the branch of the state where all of the control qubits are in state $|0\rangle$ only applies the bias rotation to the target qubits, while the branch where all of the control qubits are in state $|1\rangle$ applies the gate $R_{y;\,j}(\sum_{0=1}^{n} \theta_{ij})$ to qubit j. Overall, the action takes the form

$$U = \prod_{ij} CR_y^{i \mapsto j}(\theta_{ij}) = \sum_{\mathbf{X}} |\mathbf{X}\rangle\langle\mathbf{X}| \otimes \bigotimes_j R_{y;\,j}\left(\sum_{i=0}^{n} X_i \theta_{ij}\right). \tag{6}$$

Here, $\mathbf{X} = X_1, \cdots, X_n$ is a bit string with elements $X_i \in \{0, 1\}$, the sum runs over all such strings, and we have included $X_0 = 1$ to represent the bias term; the tensor product over j implies that the rotation on the jth qubit associated with the branch of the wavefunction where the qubits are in state $|\mathbf{X}\rangle$ is by an angle $\sum_{i=0}^{n} X_i \theta_{ij}$, corresponding to the standard factor in ANNs. This structure is schematized in the "2nd hidden layer" segment of Fig. 5b, where the connections between control and target qubits are explicit and their weights are given accordingly. All output values of the perceptron are essentially kept coherently, and the system is ready to have the process repeated in a subsequent layer.

Put another way, we have replaced the non-linear activation function σ in a standard perceptron by the controlled operations CR_y. Unlike σ, the output of CR_y is not deterministic; it is probabilistic. Nevertheless, if we measure one of the control qubits, we know we will find state $|0\rangle$ with probability $p(0) = \cos^2 \frac{z}{2}$ times and state $|1\rangle$ with the rest of the probability $p(1) = \sin^2 \frac{z}{2}$, depending on the value of $z = \sum_{i=1}^{n} X_i W_i + b$. We can turn such probabilities into binary outputs by simply choosing the larger of the two, which are split at the value $\alpha = \pi/2$:

$$\tilde{y} = \begin{cases} 1, & p(1) > p(0) \iff z > \alpha \\ 0, & p(1) < p(0) \iff z < \alpha \end{cases}. \tag{7}$$

In this sense, we have created a coherent QNN that retains all of the properties of an ANN while allowing for data to be processed without each perceptron being restricted to a binary output.

After repeating the process for multiple layers, the number of controlled (entangling) gates is given by the number of connections between the layers. This is at most a quadratic function of the number of nodes per layer and a linear function in the depth of the neural network, which are expected to grow with the number of features in the data but do not follow a fixed relationship. One can thus create a few-qubit quantum neural network with $n \ll N$ and verify empirically its success for a given machine learning tasks.

Finally, after all of the intermediate (hidden) layers, we perform a measurement and calculate the expectation value of some operator, which we further use to decide the class of the data point as in a classical NN. In our work, we often measure the value of the final (Nth) qubit $\langle Z_N \rangle$ as depicted in the "output layer" segment of Fig. 5b. The measurement result is fed into a classical non-linear activation function σ to produce the binary outcome

$$\tilde{y} = \begin{cases} 1, & \sigma(\langle Z_N \rangle) = 1 \\ 0, & \sigma(\langle Z_N \rangle) = 0 \end{cases}. \tag{8}$$

More general outcomes can be considered by either dividing the ranges of expectation values $\langle Z_N \rangle$ into more than two segments or by measuring more final qubits to yield more possible final outcomes. The results of such outcomes can be used to update the encoding weights W_i and intermediate weights θ_{ij} throughout the network in an iterative fashion.

This method can create a complete FFNN like a standard classical one without doing intermediate measurements. An overall schematic diagram of a CFFQNN with two hidden layers with four and three qubits respectively is shown in Fig. 5b.

2.1.1 Different variations of the model

In our work, we deploy two distinct versions of the CFFQNN model: the standard CFFQNN and a variant that we dub FixedCFFQNN. While FixedCFFQNN retains the architectural design of the CFFQNN, it diverges in one key aspect: the weights in its initial layer remain untrained. This reduction in the number of parameters to be trained significantly speeds the training process while still outperforming previous QNNs.

2.1.2 Hyperparameters of the CFFQNN model

The architecture of CFFQNN closely mirrors that of an ANN, sharing many of the same hyperparameters. This allows for customization in terms of the number of layers and nodes within each layer. Additionally, the measurement scheme can be tailored to fit specific data needs; even the parameter α in Eq. (8) is a hyperparameter. For instance, throughout our research, we employed a single measurement strategy for CFFQNN, measuring only the final qubit. For the FixedCFFQNN, we adopted a partial measurement approach, targeting all qubits except the qubits in the initial layer.

2.2 Numerical results

We evaluate the CFFQNN's performance against the prevailing QNN model across various datasets. For the conventional QNN model, we employ the ZZFeatureMap (contributors 2023) to encode classical data into the quantum circuit and the RealAmplitudes (contributors 2023) circuit as the variational circuit with the trainable weights. We use the COBYLA (Pedregosa et al. 2011) optimizer to optimize the weights (θs). Figure 2 illustrates a 3-qubit ZZFeatureMap circuit alongside a RealAmplitudes circuit with two repetition layers. Additionally, to provide a comprehensive performance perspective, we compared the efficacy of all quantum models against the classical MLPClassifier-an FFNN.

To predict the input data's output class, we predominantly employed the Statevector simulator from Qiskit (contributors 2023; Asfaw et al. 2020), designed to emulate the ideal quantum states of a quantum system without any external noise or decoherence. This simulator offers an exact depiction of the quantum state, facilitating precise calculations and forecasts. It proves especially valuable for theoretical investigations and grasping the optimal performance of quantum algorithms.

2.3 Data and metrics

We evaluated the efficacy of our quantum machine learning model using the credit card fraud detection and breast cancer diagnostic datasets. These datasets serve as standard benchmarks for testing and comparing different machine learning models. Details about these models can be found in Section 4.

2.4 Results on credit card dataset

For this study, we used the PCA to reduce the dimension of credit card dataset to seven features. For both CFFQNN and FixedCFFQNN, we utilized a network structure with three nodes in the initial layer, two in the subsequent layer, and a single node in the concluding layer.

Figure 6a presents a performance comparison between CFFQNN, FixedCFFQNN, QNN, and the classical MLPClassifier. The bar chart clearly demonstrates the superior performance of both CFFQNN variants over the conventional QNN model. Despite augmenting the QNN with 35

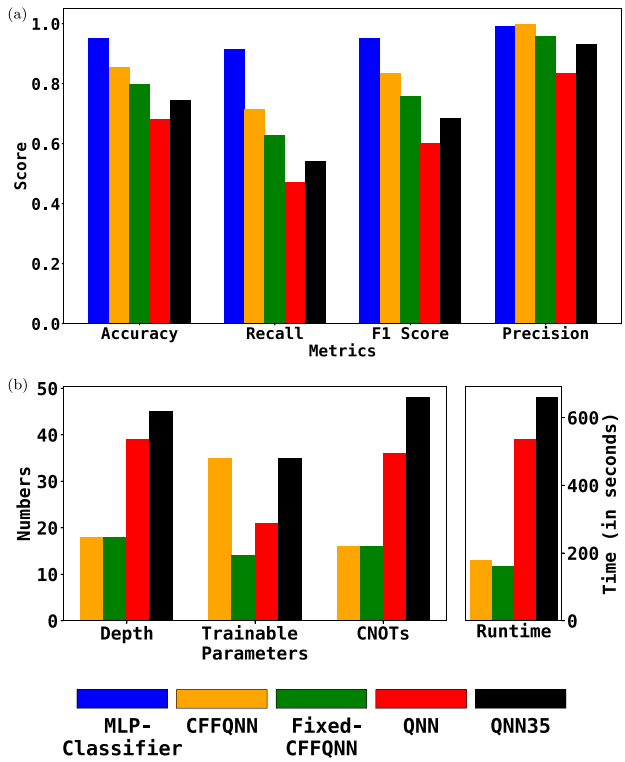

Fig. 6 **a** Performance comparison of various models on the credit card fraud detection dataset, highlighting metrics such as accuracy, recall, F1 score, and precision. Higher scores are better, showcasing the advantage of the CFFQNN over QNNs. **b** Comparative analysis of resource utilization for QNN and CFFQNN models on the same dataset. The depth, number of trainable parameters, and CNOTs are indicated by the vertical axis on the left, while the runtime is indicated by the vertical axis on the right. Note that the bars from left to right represent the MLP-Clasifier, CFFQNN, FixedCFFQNN, QNN, and QNN35, respectively. The resource utilization bar for the MLPClassifier is omitted in **b** since it is not a quantum model. Lower resource utilization is better, again demonstrating the advantages of the CFFQNN

parameters for a balanced comparison, there was no notable enhancement in its results.

Figure 6b outlines the quantum resources utilized by each quantum model. Evidently, the CFFQNN models require a reduced circuit depth, fewer CNOT gates, and a shorter simulator runtime compared to the QNN model. We specify the number of CNOT gates and circuit depth as resources, instead of the overall circuit complexity, because the former are fixed by the framework of the QNN alone while the latter depends on the actual values that the weight parameters inside the network take, which are varied during training. Nonetheless, we expect a close correlation between CNOT gates, circuit depth, and overall circuit complexity due to the directional nature of our QNN, where later qubits only interact with earlier qubits in one layer. Additionally, the FixedCFFQNN model exhibits a reduced requirement on the number of parameters.

2.5 Results on breast cancer dataset

For the breast cancer dataset, we narrowed down the feature space to 7 features and executed 100 iterations of COBYLA to enhance the quantum models. We adopted a network structure similar to that used for the credit card dataset in our proposed quantum models.

In Fig. 7a, it is evident that both CFFQNN and FixedCFFQNN outperform the existing QNN models. The QNN's recall, F1, and precision scores are all zero, indicating a failure of the model to effectively learn from the data, resulting in the misclassification of all test data into a single category.

Figure 7b provides a comparative analysis of the resources consumed by each of the four quantum models. While the resource utilization is largely consistent across all models, it is noteworthy that the QNN models demand a significantly higher number of CNOT gates compared to the CFFQNN variants.

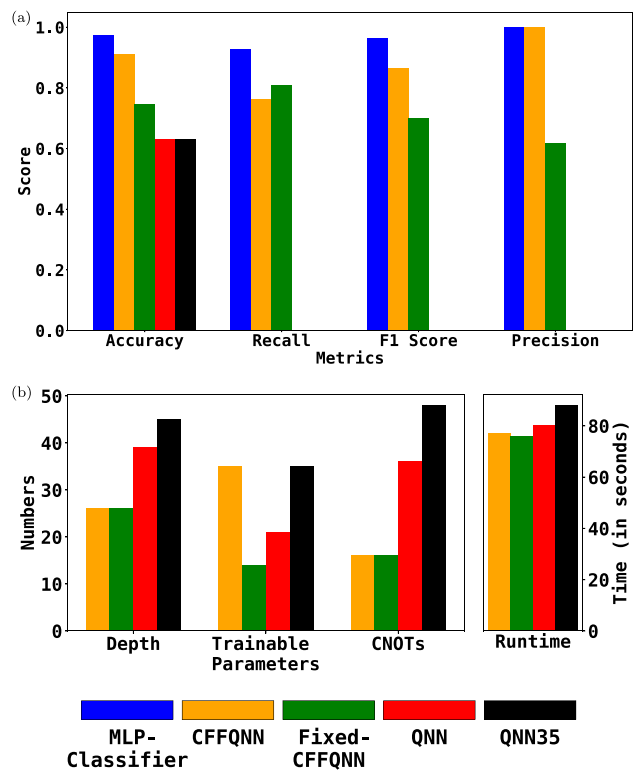

Fig. 7 **a** Performance comparison of various models on the breast cancer diagnostic dataset, highlighting metrics such as accuracy, recall, F1 score, and precision. Higher scores are better, showcasing the advantage of the CFFQNN over QNNs. **b** Comparative analysis of resource utilization for QNN and CFFQNN models on the same dataset. The depth, number of trainable parameters, and CNOTs are indicated by the vertical axis on the left, while the runtime is indicated by the vertical axis on the right. Note that the bars from left to right represent the MLP-Clasifier, CFFQNN, FixedCFFQNN, QNN, and QNN35, respectively. The resource utilization bar for the MLPClassifier is omitted in **b** since it is not a quantum model. Lower resource utilization is better, again demonstrating the advantages of the CFFQNN

2.6 Results on other datasets

To verify that the CFFQNN did not merely succeed by chance, we performed further analyses that are described in Appendix 1. For a further classification problem related to diagnosing heart disease, the CFFQNN outperforms existing QNNs by over 10 percentage points in accuracy while using half or fewer CNOT gates. For a regression task related to student performance, the mean square error of our CFFQNN is slightly better than that of existing QNNs, while our fixed CFFQNN beats both by a factor of almost 20. These display that our advantages can be broadly applicable to many tasks that involve neural networks.

3 Discussion and conclusion

Our approach and results demonstrate a step forward in quantum machine learning through the introduction of the CFFQNN. Our CFFQNN is the direct quantum upgrade of a classical ANN, with all possible outputs of each classical perceptron upgraded to a branch of the quantum state that exists simultaneously and interacts with all other branches. The advantages of the CFFQNN over previous QNNs stem from its unique integration of the $\sum_i X_i W_i + b$ term, inspired by classical neural networks, into the initial layer within a higher-dimensional Hilbert space. This approach, combined with the strategic incorporation of quantum entanglements in subsequent layers, marks a significant advancement over traditional QNN models.

The CFFQNN inherits the flexibility and empirical nature of classical ANNs. Depending on the size of the data and its particular features, different numbers of hidden layers and nodes therein are optimal for training without overfitting (see Appendix 2 for an analysis of the non-linear impact of changing numbers of layers on model performance). Just as there is no hard-and-fast rule for specifying the optimal size of a classical ANN, our model should be tuned for the task at hand and has been successful for all of the tasks described here.

A pivotal advantage of our model is that its qubit count and number of CNOT gates in its circuit are independent from the number of features in the dataset. This independence is particularly advantageous in dealing with complex, realistic datasets with varying feature sizes, ensuring scalability and flexibility. Furthermore, the model's design allows for direct adjustments in intermediate layers, a feature that enables it to effectively support deep network architectures, a limitation in many existing QNN models. These low resource counts make the CFFQNN suitable for quantum hardwares that exist around the world, and we look forward to an experimental demonstration of the CFFQNN in the near term.

Through numerical experimentation, we have demonstrated the superior performance of CFFQNN in classification tasks, including a notable variant: the FixedCFFQNN. In the latter approach, we did not train any parameters in the first layer, yet it still outperformed existing QNN models. This untrained variant underscores the inherent efficiency and robustness of the CFFQNN architecture. Achieving high accuracy while requiring minimal quantum resources, both the standard and Fixed CFFQNN variants are significant steps toward the practical use of quantum machine learning. Compared to QNN models utilizing the ZZFeature Map, the CFFQNN exhibits not only enhanced performance but also a more efficient utilization of quantum resources. These characteristics of our model will persuade realization on various quantum computing hardware as initial steps toward a scalable implementation for the practical application of quantum machine learning.

4 Methods

We create an instance of the CFFQNN to perform data classification on two standard datasets. The CFFQNN model is simulated using Qiskit, while the datasets employed are accessible through Kaggle. We here detail parameters and techniques used in creating, training, and evaluating the model.

First, we select the maximum number of qubits to be used in our network as seven: this is small enough to be simulable on a standard personal computer yet large enough to exhibit genuinely quantum features such as a large Hilbert space spanned by 128 elements. The major question is whether this is a sufficient number of qubits to perform nontrivial machine learning tasks. Standard QNNs require one qubit per feature in the dataset, so we limit our investigation to data with 7 features. In comparison, the CFFQNN can handle more features with the same total number of qubits, so we note that the number of features is limited by the QNNs against which we seek to compare the CFFQNN, not by the CFFQNN itself.

Next, we take real-world datasets and reduce their feature spaces to make the data size suitable for simulation, noting that many-qubit quantum systems are inherently challenging to simulate and that therein lies potential quantum advantages. The Credit Card Fraud detection dataset has 30 features with which one seeks to evaluate whether a given transaction was fraudulent or not, a binary classification problem, while the Breast Cancer Diagnostic dataset's 30 features are used to evaluate whether a given patient does or does not have breast cancer, another binary classification. Since some of the features may be highly correlated with each other or may contribute less to the overall variance in the data distribution, a linear transformation of the coordinates in the

feature space can elicit the principal components, which are the new coordinate axes that account for most of the independent information contained in the features and allow one to neglect axes where the data change less. Such principal component analysis (PCA) is standard in data processing and here reduces both 30-feature datasets to seven principal features each. These details are summarized in Table 1.

In the case of the Credit Card dataset, we also addressed the issue of class imbalance. To ensure unbiased training and evaluation, we eliminated the excess class instances, balancing the dataset. This step enabled our model to learn from both the minority and majority classes more effectively, thereby enhancing its ability to detect fraudulent transactions accurately. By employing these preprocessing techniques on the selected datasets, we aimed to create a robust and reliable framework for evaluating the effectiveness of our quantum machine learning model.

The standard QNN is programmed as follows. Every qubit is initialized in the superposition state $(|0\rangle + |1\rangle)/\sqrt{2}$ by means of a Hadamard transformation, and then the data features are encoded using a phase gate

$$P(\bar{X}_i) = |0\rangle\langle 0| + \mathrm{e}^{\mathrm{i}\bar{X}_i}|1\rangle\langle 1| \tag{9}$$

acting on the ith qubit; this is the ZFeatureMap with $\bar{X}_i = 2X_i$ and is the first stage of the ZZFeatureMap.

The ZZFeatureMap then continues to sequentially entangle the $i$the and jth qubits and again upload the same data onto the quantum state, using the sequence of gates:

$$G_{ij} = \mathrm{CNOT}^{i\mapsto j}[\mathbb{I}_i \otimes P_j(X_{ij})]\mathrm{CNOT}^{i\mapsto j} \tag{10}$$

which uses the gate $\mathrm{CNOT}^{i\mapsto j} = (|0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes \sigma_x)$ and the non-linear function of the parameters $X_{ij} = 2(\pi - X_i)(\pi - X_j)$. All of the qubits are pairwise entangled using a sequence of G_{ij} operators for various i and j. However, the relationship between the number of CNOT gates and the number of qubits is not fixed in a strict mathematical sense, but rather it depends on the specific architecture of the ZZFeatureMAp quantum circuit and the requirements of the algorithm being implemented. Here, we used the circuit with *full* entanglement option, which requires $\frac{N(N-1)}{2}$ CNOT gates to fully entangle all pairs of qubits for single repetition of the circuit.

After all of the features are uploaded, the next step of the QNN is a parametrized quantum circuit. A single repetition of this consists of at least $2N$ single-qubit rotation gates $R_y(\theta_i)$, N acting on each qubit, separated by fixed entangling gates. Each qubit experiences one parametrized R_y gate, then a sequence of entangling gates $\prod_{i=1}^{i-1}\mathrm{CNOT}^{N-1\mapsto N}\cdots\mathrm{CNOT}^{2\mapsto 3}\mathrm{CNOT}^{1\mapsto 2}$ is applied, and then the process is repeated in alternating fashion and ends with parametrized single-qubit rotation gates for a total of $2(N-1)$ controlled operations in the parametrized circuit. All the qubits are then measured in the computational basis, and the measurement result is processed in the same way as for the CFFQNN detailed below.

In comparison, the data may be encoded into any number of qubits for the CFFQNN, with more qubits being required for subsequent manipulations that correspond to hidden layers of ANNs. Just like in classical machine learning, there is no *a priori* method for determining how many layers and how many nodes in each layer will be required for the success of training the network for a particular dataset. We choose to encode our datasets' seven features into three qubits, corresponding to the first hidden layer, process them with a second hidden layer comprised of two qubits, and then funnel the quantum information into a final qubit such that the total number of qubits is only six.

The three qubits have the same data redundantly uploaded into them using no entangling operations: the operator $R_y(\sum_{i=1}^{N} W_i X_i + b)$ is applied to each of the first three qubits as in the main text. To process the data and forward it to the second hidden layer, a controlled operation is required between each pair of qubits from the first and second layers, such that 12 operations of the form $CR_y^{i\mapsto j}(\theta_{ij})$ with unique parameterized weights θ_{ij} are applied. A single-qubit rotation corresponding to a biasing term is also applied to each qubit in the second layer. Finally, three controlled operations $CR_y^{j\mapsto N}(\theta_{jN})$ are performed between the qubits in the second hidden layer and the final qubit along with a biasing term on the final qubit, for a total of 16 controlled operations. The final qubit is measured in the computational basis.

For both setups, the single output parameter $\langle Z_N\rangle$ is fed into a classical non-linear function and used to classify the input data. At least 70% of the available data points are used, and the models are scored on how well they correctly predict the classification of those data. The parameterized circuits

Table 1 Properties of the datasets used to evaluate the CFFQNN and compare it to existing neural networks

Datasets	Features	Features used	Training size	Testing size	Labels
Credit card fraud detection (balanced)	30	7	688	296	2
Breast cancer diagnostic (Wisconsin)	30	7	455	114	2

Table 2 Metrics used to evaluate and compare the performances of different neural networks on the same datasets

Metrics	Equations
Precision	$\frac{TP}{TP+FP}$
Recall	$\frac{TP}{TP+FN}$
Accuracy	$\frac{TP+TN}{TP+FP+FN+TN}$
F1 score	$\frac{2\times Precision\times Recall}{Precision+Recall}$

are then updated with new parameters obtained from the COBYLA (Pedregosa et al. 2011) optimizer, and this process is repeated iteratively until the parameters converge. Those parameters are then used to test the models' performances on the test data points that they have not seen before. The overall models are then evaluated using the numbers of true positive (TP), false positive (FP), true negative (TN), and false negative (FN) results. These can be combined into a number of metrics as in Table 2. All of these metrics range from zero to one, with larger numbers implying superior models.

All of the source code for creating and comparing these models is detailed in a GitHub repository and will be available upon request.

Appendix 1. Additional results

This section provides additional performance results comparing our proposed quantum neural network (QNN) model with an existing QNN model that uses a ZZFeatureMap and a RealAmplitude circuit.

Heart disease dataset

The Heart Disease dataset, sourced from Kaggle (LAPP 2024), contains 918 instances with 13 features used to predict the presence of heart disease. To enhance the model's performance and due to limited resources for running quantum simulations, principal component analysis (PCA) was applied to reduce the number of features to 7, capturing the most significant variance in the data.

The existing QNN model achieved an accuracy of 65.76%. In contrast, our proposed CFFQNN model demonstrated superior performance with an accuracy of 78.80% when using a layer structure of [2,3,1]. Even with a simpler layer structure of [1,1,1], our model achieved an accuracy of 76.09%.

In terms of circuit complexity, the existing QNN model has a circuit depth of 39 and utilizes 36 CNOT gates. Our proposed model, with a layer structure of [2,3,1], achieved a reduced circuit depth of 26 with 18 CNOT gates. Furthermore, with a layer structure of [1,1,1], our model significantly reduced the circuit depth to 14 and required only 4 CNOT gates.

These results highlight the enhanced efficiency and effectiveness of our CFFQNN model, demonstrating both higher accuracy and reduced circuit complexity compared to the existing model.

Student performance dataset (linear regression)

We further evaluated our proposed quantum neural network models using the student performance dataset from Kaggle (Narayan 2024), focusing on regression tasks. The mean square error (MSE) values for different models are as follows: our proposed CFFQNN achieved an MSE of 0.1869, the fixed CFFQNN achieved a significantly lower MSE of 0.0102, and the existing QNN model resulted in an MSE of 0.1982. These results demonstrate the superior performance of the fixed CFFQNN model in regression tasks, indicating its ability to generalize better compared to both the CFFQNN and the existing QNN models.

In terms of computational resources, the depth, number of parameters, and number of CNOT gates for each model are as follows: the CFFQNN has a depth of 20, 21 parameters, and 12 CNOT gates; the fixed CFFQNN has a similar depth of 20 but with only 11 parameters and 12 CNOT gates; and the existing QNN model has a greater depth of 31, with 15 parameters and 24 CNOT gates. The resource efficiency of the fixed CFFQNN, with fewer parameters and a reduced number of CNOT gates while maintaining a lower circuit depth, highlights its computational advantage over the existing QNN model.

Appendix 2. Impact of model depth

Resource limitations

In our study, we utilized a dataset with only 7 features due to resource constraints. To conduct a thorough analysis of the CFFQNN model's depth, it is necessary to use larger datasets, which require computational resources that we currently do not have access to. Training deeper models on such datasets demands significant computational power and memory. Moreover, even if we were able to perform such an analysis, it is not guaranteed that increasing the model depth would result in improved performance. This is a phenomenon observed in classical neural network structures as well.

Non-trivial relationship between depth and performance

The relationship between model depth and performance is complex and not necessarily linear. Increasing the number of layers in a neural network does not always result in improved performance. For instance, He et al. (2016) introduced the

concept of residual networks (ResNets), demonstrating that simply adding more layers can lead to training degradation due to the vanishing gradient problem. Similarly, Li et al. (2016) highlighted that deeper models can suffer from overfitting, and their performance may plateau or even degrade beyond a certain depth.

In our specific model, we observed similar trends. For example, our proposed QNN model with a layer structure of [2,3,1] achieved the highest accuracy of 0.7880 on the Heart disease dataset. Interestingly, even a simpler structure of [1,1,1] achieved a notable accuracy of 0.7609. However, increasing the complexity to structures like [3,1,1] and [3,2,1] resulted in lower accuracies of 0.5978 and 0.6359, respectively. The detailed performance of various layer structures is summarized in Table 3.

These results highlight the non-linear relationship between depth and performance, illustrating that increased model depth does not necessarily equate to better performance. Additionally, in our specific model, performance is influenced by initial points and the type of measurement used. These factors further complicate the relationship between depth and performance.

Practical approaches to determining model depth

In the absence of a fixed methodology to determine the optimal number of layers, it is common practice in neural network design to rely on heuristic and empirical approaches. Practitioners often start with their experience and intuition when designing a model, iterating through different configurations and systematically testing various hyperparameters to find an optimal setup. Goodfellow et al. (2016) suggest that the architecture of deep learning models often involves iterative experimentation and validation to find a suitable balance between model complexity and performance. This iterative process involves starting with a baseline model and gradually increasing complexity while monitoring validation performance. These methodologies are valid for our CFFQNN model as well, where initial experimentation and validation play a crucial role in finding the optimal depth and configuration.

Table 3 Results for heart disease dataset for different layers

Layers	Accuracy
[1, 1, 1]	0.760
[1, 2, 1]	0.722
[1, 3, 1]	0.679
[1, 4, 1]	0.766
[2, 1, 1]	0.750
[2, 2, 1]	0.760
[2, 3, 1]	**0.788**
[3, 1, 1]	0.597
[3, 2, 1]	0.635
[3, 3, 1]	0.750

Acknowledgements The authors acknowledge that the NRC headquarters is located on the traditional unceded territory of the Algonquin Anishinaabe and Mohawk people. The authors would like to acknowledge the use of IBM Quantum services for this work and in particular the Qiskit package (contributors 2023). The authors thank Farid Ghobadi and Barry Sanders for insightful discussions. AZG acknowledges support from NSERC's postdoctoral fellowship. KH acknowledges support from NSERC's Discovery Grant program.

Author Contribution US conceived the model and developed numerical tests for performance analysis and comparison. All authors reviewed the results and developed the overall description and conclusions of the proposed model. All authors contributed to the writing and editing of the manuscript.

Funding Open access funding provided by National Research Council Canada library.

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of Interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Abbas A, Sutter D, Zoufal C, Lucchi A, Figalli A, Woerner S (2021) The power of quantum neural networks Nat. Comput Sci 1:403. https://doi.org/10.1038/s43588-021-00084-1

Asfaw A, Bello L, Ben-Haim Y, Bravyi S, Capelluto L, Vazquez AC, Ceroni J, Chen R, Frisch A, Gambetta J, Garion S, Gil L, Gonzalez SDLP, Harkins F, Imamichi T, McKay D, Mezzacapo A, Minev Z, Movassagh R, Nannicini G, Nation P, Phan A, Pistoia M, Rattew A, Schaefer J, Shabani J, Smolin J, Temme K, Tod M, Wood S (2020) Learn quantum computation using Qiskit. https://qiskit.org/textbook/preface.html

Beer K, Bondarenko D, Farrelly T, Osborne TJ, Salzmann R, Scheiermann D, Wolf R (2020) Training deep quantum neural networks Nat. Commun 11:1. https://doi.org/10.1038/s41467-020-14454-2

Bharti K, Cervera-Lierta A, Kyaw TH, Haug T, Alperin-Lea S, Anand A, Degroote M, Heimonen H, Kottmann JS, Menke T, Mok W-K, Sim S, Kwek L-C, Aspuru-Guzik A (2022) Noisy intermediate-scale quantum algorithms. Rev Mod Phys 94:015004. https://doi.org/10.1103/RevModPhys.94.015004

Biamonte J, Wittek P, Pancotti N, Rebentrost P, Wiebe N, Lloyd S (2017) Quantum machine learning. Nature 549:195–202. https://doi.org/10.1038/nature23474

Bondarenko D, Feldmann P (2020) Quantum autoencoders to denoise quantum data Phys. Rev Lett 124:130502. https://doi.org/10.1103/PhysRevLett.124.130502

Cerezo M, Verdon G, Huang H-Y, Cincio L, Coles PJ (2022) Challenges and opportunities in quantum machine learning. Nat Comput Sci 2:567. https://doi.org/10.1038/s43588-022-00311-3

Chalumuri A, Kune R, Manoj BS (2021) A hybrid classical-quantum approach for multi-class classification. Quantum Inf Process 20:1. https://doi.org/10.1007/s11128-021-03029-9

Chen B-Q, Niu X-F (2020) Quantum neural network with improved quantum learning algorithm. Int J Theor Phys 59:1978. 10.1007/s10773-020-04470-9

Cong I, Choi S, Lukin MD (2019) Quantum convolutional neural networks Nat. Phys 15:1273. https://doi.org/10.1038/s41567-019-0648-8

contributors Q (2023) Qiskit: an open-source framework for quantum computing. https://doi.org/10.5281/zenodo.2573505

Cybenko G (1989) Approximation by superpositions of a sigmoidal function. Math Control Signals Systems 2:303. https://doi.org/10.1007/BF02551274

Diep DN (2020) Some quantum neural networks. Int J Theor Phys 59:1179. https://doi.org/10.1007/s10773-020-04397-1

Farhi E, Neven H (2018) Classification with quantum neural networks on near term processors. Quantum Phys

García DP, Cruz-Benito J, García-Peñalvo FJ (2022) Systematic literature review: quantum machine learning and its applications. https://doi.org/10.48550/arXiv.2201.04093

Goodfellow I, Bengio Y, Courville A (2016) Deep learning (publisher MIT Press). http://www.deeplearningbook.org

Gyurik C, Molteni R, Dunjko V (2023) Limitations of measure-first protocols in quantum machine learning. arXiv:2311.12618 [quant-ph]

Havlíček V, Córcoles AD, Temme K, Harrow AW, Kandala A, Chow JM, Gambetta JM (2019) Supervised learning with quantum-enhanced feature spaces. Nature 567:209. https://doi.org/10.1038/s41586-019-0980-2

He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: IEEE Conference on computer vision and pattern recognition (CVPR) (IEEE) pp 27–30. https://doi.org/10.1109/CVPR.2016.90

Hornik K (1991) Approximation capabilities of multilayer feedforward networks. Neural Netw 4:251. https://doi.org/10.1016/0893-6080(91)90009-T

Jiang W, Xiong J, Shi Y (2021) A co-design framework of neural networks and quantum circuits towards quantum advantage. Nat Commun 12:1. https://doi.org/10.1038/s41467-020-20729-5

LAPP D (2024) Heart Disease Dataset. Accessed 11 Jul 2024. https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

Li L, Jamieson K, DeSalvo G, Rostamizadeh A, Talwalkar A (2016) Hyperband: a novel bandit-based approach to hyperparameter optimization. https://doi.org/10.48550/arXiv.1603.06560

Li Y, Zhou R-G, Xu R, Luo J, Hu W (2020) A quantum deep convolutional neural network for image recognition. Quantum Sci Technol 5:044003. https://doi.org/10.1088/2058-9565/ab9f93

Narayan N (2024) Student performance (multiple linear regression). Accessed 11 Jul 2024. https://www.kaggle.com/datasets/nikhil7280/student-performance-multiple-linear-regression/data

Nielsen MA, Chuang IL (2011) Quantum computation and quantum information: 10th Anniversary Edn 10th ed. (Cambridge University Press USA)

Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E (2011) Scikit-learn: machine learning in python. J Mach Learn Res 12:2825

Preskill J (2018) Quantum Computing in the NISQ era and beyond Quantum 2:79. arXiv:1801.00862v3. https://doi.org/10.22331/q-2018-08-06-79

Rosenblatt F (1957) The perceptron, a perceiving and recognizing automaton Project Para, Report: Cornell Aeronautical Laboratory (Cornell Aeronautical Laboratory). https://books.google.ca/books?id=P_XGPgAACAAJ

Sagingalieva A, Kordzanganeh M, Kenbayev N, Kosichkina D, Tomashuk T, Melnikov A (2023) Hybrid quantum neural network for drug response prediction. Cancers 15. https://doi.org/10.3390/cancers15102705

Schuld M, Petruccione F (2021) Machine learning with quantum computers. (Springer International Publishing. Cham, Switzerland)

Schuld M, Sinayskiy I, Petruccione F (2014) An introduction to quantum machine learning. Contemp Phys 56:172. https://doi.org/10.1080/00107514.2014.964942

Sharma K, Cerezo M, CincioP L, Coles J (2022) Trainability of dissipative perceptron-based quantum neural networks Phys. Rev Lett 128:180505. https://doi.org/10.1103/PhysRevLett.128.180505

Sim S, Johnson PD, Aspuru-Guzik A (2019) Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms Adv. Quantum Technol 2:1900070. https://doi.org/10.1002/qute.201900070

Situ H, He Z, Wang Y, Li L, Zheng S (2018) Quantum generative adversarial network for generating discrete distribution. https://doi.org/10.1016/j.ins.2020.05.127. arXiv:1807.01235

Tacchino F, Mangini S, Kl P, Barkoutsos C, Macchiavello D, Gerace I, Tavernelli D. Bajoni (2021) Variational learning for quantum artificial neural networks. IEEE Trans Quantum Eng 2:1. https://doi.org/10.1109/TQE.2021.3062494

Tacchino F, Barkoutsos P, Macchiavello C, Tavernelli I, Gerace D, Bajoni D (2020) Quantum implementation of an artificial feed-forward neural network. Quantum Sci Technol 5:044010. https://doi.org/10.1088/2058-9565/abb8e4

Wan KH, Dahlsten O, Kristjánsson H, Gardner R, Kim MS (2017) Quantum generalisation of feedforward neural networks npj Quantum Inf 3:1. https://doi.org/10.1038/s41534-017-0032-4

Wang J, Chen Y, Chakraborty R, Yu SX (2019) Orthogonal convolutional neural networks. https://doi.org/10.48550/arXiv.1911.12207

Wu SL, Sun S, Guan W, Zhou C, Chan J, Cheng CL, Pham T, Qian Y, Wang AZ, Zhang R, Livny M, Glick J, KlBarkoutsos P, Woerner S, Tavernelli I, Carminati F, Di Meglio A, Li ACY, Lykken J, Spentzouris P, Chen SY-C, Yoo S, Wei T-C (2021) Application of quantum machine learning using the quantum kernel algorithm on high energy physics analysis at the LHC. https://doi.org/10.1103/PhysRevResearch.3.033221. arXiv:2104.05059

Yamasaki H, Isogai N, Murao M (2023) Advantage of quantum machine learning from general computational advantages. https://doi.org/10.48550/arXiv.2312.03057

Zhang S-X, Allcock J, Wan Z-Q, Liu S, Sun J, Yu H, Yang X-H, Qiu J, Ye Z, Chen Y-Q, Lee C-K, Zheng Y-C, K.Jian S, Yao H, Hsieh C-Y, Zhang S (2023) TensorCircuit: a quantum software framework for the NISQ era. Quantum 7:912. arXiv:2205.10091v2. https://doi.org/10.22331/q-2023-02-02-912

Zhou M-G, Liu Z-P, Yin H-L, Li C-L, Xu T-K, Chen Z-B (2023) Quantum neural network for quantum neural computing. Research 6. https://doi.org/10.34133/research.0134

Zoufal C, Lucchi A, Woerner S (2019) Quantum generative adversarial networks for learning and loading random distributions. npj Quantum Inf 5:1. https://doi.org/10.1038/s41534-019-0223-2

Zoufal C, Lucchi A, Woerner S (2021) Variational quantum Boltzmann machines. Quantum Machine Intelligence 3. https://doi.org/10.1007/s42484-020-00033-7

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.